

Simulado – Conceitos básicos de testes de unidade e JUnit

1. Uando boas práticas de testes de unidade, quantos métodos podem ser testados com um teste de unidade?

- a) Apenas um método pode ser testado
- b) No máximo dois métodos por teste
- c) No máximo cinco métodos por teste
- d) JUnit limita a 10 métodos por teste
- e) Um teste de unidade pode testar quantos métodos forem necessários

2. Quantos testes de unidade separados devem ser criados para testar um único método?

- a) Apenas um teste
- b) Exatamente dois testes: um positivo e um negativo
- c) Exatamente cinco testes, conforme recomendação da Oracle
- d) Apenas um teste, desde que utilize várias asserções
- e) Quantos forem necessários para cobrir os cenários relevantes daquele método

3. Qual é a principal diferença entre Teste de Unidade e Teste de Integração?

- a) Testes de unidade sempre verificam a interface gráfica
- b) Testes de integração testam código externo, enquanto testes de unidade ignoram o código interno
- c) Testes de unidade verificam métodos isoladamente; testes de integração verificam métodos em conjunto com suas dependências
- d) Testes de unidade devem usar banco de dados real; integração nunca usa banco real
- e) Testes de unidade precisam de *mocks*, enquanto testes de integração nunca podem usá-los

4. Se o método em teste (SUT) depende de um objeto de outra classe, como essa dependência deve ser disponibilizada ao método?

- a) Usando injeção de dependências para fornecer o objeto de outra classe
- b) Criando o objeto manualmente dentro do método sob teste usando `new`
- c) Ignorando a dependência e testando apenas o método principal
- d) Pedindo ao usuário para informar a dependência por entrada de dados
- e) Adicionando a dependência no mesmo arquivo do método

5. Que código devemos testar com testes de unidade?

- a) Devemos testar código da API Java para garantir compatibilidade
- b) Devemos testar bibliotecas externas que não foram escritas por nós
- c) Devemos testar todos os métodos usados pelo método sob teste, mesmo os que vêm de *frameworks*
- d) Devemos testar apenas o código que nós escrevemos na aplicação
- e) Devemos testar apenas métodos privados

6. Qual é o principal objetivo de inserir uma mensagem de asserção nos testes JUnit?

- a) Acelerar a execução dos testes
- b) Fornecer informação adicional e clara sobre o motivo da falha do teste
- c) Melhorar o desempenho em ambiente de produção
- d) Substituir a descrição do método de teste
- e) Aumentar a prioridade do teste no relatório final

7. Sobre o comportamento padrão do JUnit ao instanciar classes de teste (SUT), qual das afirmações a seguir é verdadeira?

- a) O JUnit cria apenas uma única instância da classe de teste (SUT) e a reutiliza em todos os métodos
- b) O JUnit compartilha a instância da classe de teste entre threads paralelas
- c) O JUnit nunca cria novas instâncias automaticamente
- d) O JUnit cria uma nova instância da classe de teste (SUT) somente no início da aplicação
- e) O JUnit cria uma nova instância da classe de teste (SUT) para cada método de teste executado

8. Sobre a anotação `@AfterAll`, qual das alternativas abaixo descreve corretamente seu comportamento?

- a) `@AfterAll` é executado uma única vez após todos os métodos de teste da classe serem finalizados
- b) `@AfterAll` é executado após cada método de teste individual
- c) `@AfterAll` reinicia a aplicação antes da execução dos testes
- d) `@AfterAll` roda antes de todos os testes serem executados
- e) `@AfterAll` é executado uma vez por método da classe de teste

9. Sobre a anotação `@AfterEach`, qual alternativa descreve corretamente seu funcionamento?

- a) `@AfterEach` é executado apenas uma vez após toda a classe de teste
- b) `@AfterEach` é executado antes de cada método anotado com `@Test`
- c) `@AfterEach` é executado após cada método anotado com `@Test`
- d) `@AfterEach` só funciona em classes anotadas com `@BeforeAll`
- e) `@AfterEach` funciona apenas se houver *mocks*

10. Qual das alternativas apresenta uma afirmação VERDADEIRA sobre a biblioteca JUnit e seus comportamentos?

- a) A anotação `@AfterEach` garante que o método anotado será executado uma única vez ao final da execução de todos os testes da classe.
- b) Métodos anotados com `@AfterAll` são executados antes de cada método de teste.
- c) O JUnit compartilha a mesma instância da classe de teste (SUT) entre os métodos para economizar memória.
- d) O JUnit 5 executa cada método de teste dentro de sua própria transação de execução, garantindo isolamento completo entre os testes por meio da criação de uma nova instância da classe de teste para cada método.
- e) A anotação `@BeforeAll` só pode ser aplicada a métodos não estáticos.